Practical Exercise: Prime Number Calculations

High Performance Computing

Nuno Lopes
(nlopes@ipca.pt)

Objective

- Develop a program that allows to calculate how many prime numbers exist in a range from 1 to N.
- Input: N, size of the set of numbers to check
- Output: number of prime numbers in the interval 1..N
- A prime number has "two and only two divisors, itself and the unit".

Trivial Function isPrime()

```
int isPrime(long number) {
  if (number < 2) return 0;
  for (long i=2; i<number; i++)
    if (number % i == 0) return 0;
  return 1;
}
```

Exercise 1 - Sequential

- Based on the previous function, develop a program that can calculate the number of primes that exist in the interval between 1 and N (e.g. 100000).

Exercise 1 - Sequential

```
int main(int argc, char** argv) {
    int N;
    int contaPrimos = 0;
    printf("Calculo de intervalo de primos\n");
    N = 10000;
    printf("Tamanho do intervalo: 1 .. %d\n", N);
    for(int i=1; i<=N; i++)
        if( isPrime(i) ) contaPrimos++;
    printf("Contagem de primos: %d\n", contaPrimos
);
    return (0);
}
```

Exercise 2 - Parallel

- Based on the sequential example, build a parallel version.
- Most of the time is spent on the for cycle, which will be the candidate for parallelisation.
- Use the Foster Methodology:
  - Partitioning: process to divide computation and data into groups.
  - Communication: determine the pattern of information exchange and synchronization between groups of tasks.
  - Agglomeration: joining task groups in order to adapt to the system architecture.
  - Mapping: assigning task groups to specific physical nodes.

Exercise 3 - Shared Variables

- Identify global variables shared in the parallel code section.
- Identify critical sections.
- Protect these sections with mutual exclusion mechanisms.

Exercise 4 - Local Temporary Variables

- Change the code structure to make use of local variables that accumulate temporary results per thread.
- Accumulate these results at the end of the execution of each thread with a global variable.
- Make use of a global variable to reduce local calculations.